Routing Is Least Learnable Where It Is Most Valuable: Bounds on Representation Routing for Web Agents

Anonymous ACL submission

Abstract

A web agent can read a page as text, look at a screenshot, or do both, and deployments usually fix one choice for every task. We measure that choice across three sites of VisualWebArena and WebArena with four vision-language backbones, using screenshot-free variants as controls and rerunning identical conditions to see which differences survive. Three findings separate questions that are usually merged. First, no observation choice is best everywhere, and the screenshot has a real, reproducible value, predictable in advance on one site: a zero-cost rule over the task text picks out classifieds tasks on which adding the screenshot alone raises success by 25–30 points for the two strongest backbones (11 for a 4B model), and for the 235B backbone the gap reappears on a full rerun. Second, which representation suits which task is a reproducible property of the task, beyond an additive model of task difficulty, yet single-run labels measure it with reliability 0.31–0.50 and, over six modes, flip most where the choice can change success, as that model predicts. Third, two lightweight routers that choose a representation per task from pre-run features, with their operating point fixed on training data, show no reliable gain over the fixed choices and their random mixtures. We describe the rerun-controlled protocol that keeps these three questions apart.

1 Introduction

An agent that operates a website has to perceive it, and the two standard ways are complementary in principle: the page's text (an accessibility tree) is precise, and a screenshot shows what the text omits. A third option fuses them, overlaying numbered marks on the screenshot and listing the marked elements as text (Yang et al., 2023; Koh et al., 2024). Practitioners typically pick one of the three for a whole deployment. The question this paper asks is the one that choice raises: *is the choice worth making per task, and can it be made?*

Answering it requires keeping apart three quantities that a single leaderboard number merges:

1. **Representation value**: does the screenshot change success, holding everything else fixed?
2. **Preference reliability**: when one representation beats another on a task, is that a stable property of the task, or a property of one run?
3. **Routing gain**: can a policy that chooses per task, using only what is known before it runs, beat the fixed choices?

A hindsight ceiling picks, for each task, whichever representation happened to succeed. However large, it answers none of them, because it rewards run-to-run luck as much as structure. We therefore rerun identical conditions and read each of the three against what a rerun alone changes.

Findings. (i) No choice wins everywhere: the fused mode leads clearly on classifieds, while text leads on WebArena's reddit tasks only by margins whose intervals cross zero (§3). (ii) The screenshot has a specific value that can be isolated and predicted: with the text payload and prompt held fixed, adding the screenshot raises success by 25–30 points for the two strongest backbones (11 for a 4B model) on the classifieds tasks that a zero-cost intent rule flags, against 5–10 on the other tasks; the effect reproduces on a full rerun for the 235B backbone and is larger for the stronger backbones we tested. On reddit the same rule finds nothing (§4). (iii) Per-task preferences between representations are reproducible beyond an additive difficulty model: a no-interaction null conditioned on every task's success count and on every mode's success count in each run is rejected in every fully rerun cell. Yet a single run measures them with reliability 0.31–0.50, and a decision study projects that labels

averaged over two to three runs would pass 0.5. Over all six modes, single-run labels also flip most on exactly the tasks where the choice can change success; the additive difficulty model already predicts this concentration, so any router trained on single runs inherits it. In this sense routing is least learnable where it is most valuable (§5). (iv) Two lightweight routers that fix their operating point on training data show no reliable gain over the fixed representations and their random mixtures; the one pooled hindsight opportunity that survives correction across curves (three modes, $p = 0.001$) comes from a router that only decides which tasks deserve the expensive mode, holds under billed cost only, and is not captured without looking at test outcomes (§6).

Contributions. A rerun-controlled measurement of text, screenshot and fused observation across three sites and four backbones, with screenshot-free ablations that isolate the screenshot; a matched estimate of the screenshot's value and an ex-ante rule that predicts it on one site; a test of whether per-task representation preferences are reproducible, with a decision study of how many runs a reliable label needs; and a deployable-policy comparison against fixed choices *and their random mixtures*, which is the comparison a routing claim has to win. We take this baseline from model-routing benchmarks and apply it to observation choice (§6), and distil these comparisons into a four-step protocol (§7). Findings (iii) and (iv) are the *bounds* of our title: limits on how reliably one run labels a per-task preference and on what routers trained on such labels achieve.

2 Setup

Sites and tasks. We use three sites of VisualWebArena (Koh et al., 2024): classifieds (224 scored tasks), reddit (203) and shopping (432). We add the reddit site of WebArena (Zhou et al., 2024) (104 tasks: the 106 minus the two unachievable tasks, whose reference answer is "N/A"). VWA-reddit and WA-reddit are different task sets on the same application, so they are not two independent benchmarks. Scored sets are the collected tasks minus protocol exclusions written as rules over task configuration and, for one reddit task, over agent trajectories (Appendix A).

Backbones and cells. B0 = Qwen3-VL-235B (API-served), B1 = Qwen3-VL-4B and B2 = Gemma-3-4B (both served locally), and B5 = GPT-5.6 (API, classifieds only). A *cell* is one site $\times$ backbone pair; the eight core cells, each run in every observation mode, are VWA-classifieds and VWA-reddit $\times$ {B0, B1, B2} and WA-reddit $\times$ {B0, B1}. B5-classifieds and the two shopping cells extend them (§4, §6) and are never pooled with the eight.

Observation modes. The deployment question has three answers: **DOM** (the page's accessibility tree as text), **Vision** (the raw screenshot, with an empty text payload) and **SoM** (a set-of-marks screenshot plus the list of marked elements as text; Yang et al., 2023). Three further modes carry no screenshot and serve as controls; the one used throughout is **P-SoM**, which is SoM with the screenshot removed. Marks and prompt are the same, so SoM−P-SoM isolates the image (Appendix C lists all six). "Text" modes withhold the *page* screenshot only: a reference image that is part of the task ("find this item") is given in every mode.

One plain agent. One model call per step emits a thought, an action and its arguments; the prompt holds the instruction, the mode's system prompt, the last eight steps and the current observation. There is no planner, memory, reflection or retry policy, so conditions differ in the observation and in how actions on it reach the browser: lacking element ids, Vision clicks by coordinates, which succeed less often in this harness than element-id clicks. Decoding is greedy, the viewport is 1280×720, the text observation covers only the current viewport and is never truncated, and the nominal budget is 30 steps (a parser defect let some extra actions through, mostly for GPT-5.6; Appendix E). Appendix B gives the full contract.

Reruns. We reran identical conditions, 27 registered same-condition pairs in all. In three cells (VWA-classifieds B0, VWA-reddit B0 and WA-reddit B1), every mode was run twice. A rerun uses the same tasks and code on freshly reset sites, so what it changes is what one run of the agent does not determine; in our data that is between 0 and 14% of outcomes, depending on mode and cell (Appendix F). Sites are reset before every run; within a run only classifieds resets between tasks, so reddit and shopping carry state from earlier episodes (Appendix E).

Scoring and cost. The evaluators are binary. Cost is the per-episode cost in US cents: the API

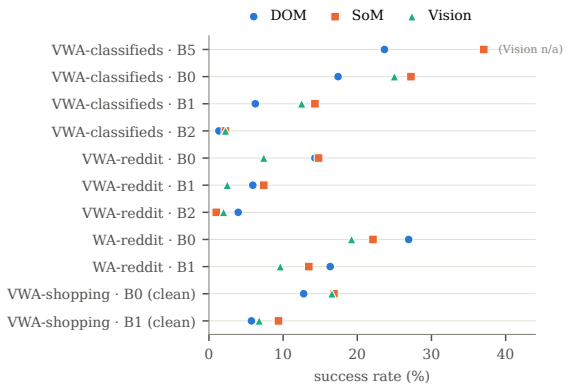


Figure 1: Success rate of the three deployment choices per cell. Shopping cells drop the tasks touched by the search-box defect in any mode and 46 state-sensitive tasks (Appendix E); B5's Vision run is excluded for a coordinate-contract defect. Sources: `routing_three_arm`, `routing_extension_cells`.

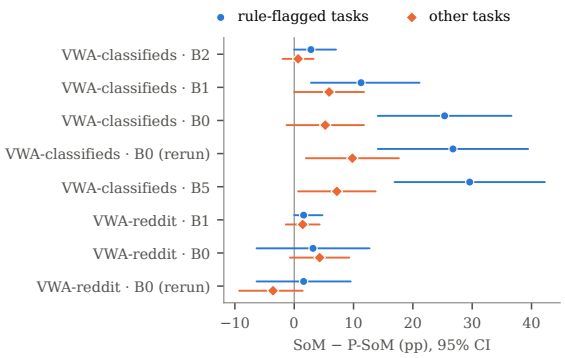


Figure 2: Value of the screenshot alone (SoM−P-SoM: same marks, same prompt) on the tasks a zero-cost intent rule flags, against the other tasks. Classifieds is out-of-sample for the rule. "Rerun" is a full repeat of both modes. VWA-reddit B2 is omitted: neither SoM nor P-SoM solves a flagged task. Source: `visual_intent_routing`.

charge for B0 and B5, and for B1 and B2 a token-priced proxy for local serving (hardware amortisation plus electricity, charged per input and output token; per-cell values in Table 4). The two kinds are ordered within a cell, never pooled across; §6 repeats the routing analysis under GPU-time and wall-clock cost.

3 No observation choice is best everywhere

Figure 1 shows the three deployment choices side by side: the fused mode's lead over text is clear on classifieds, while text's leads on reddit all have intervals crossing zero once inherited credit is removed. On classifieds the fused mode is the best of the three for B0 and B1 and beats DOM for GPT-5.6, whose Vision run is excluded (B2 solves too few tasks to order the modes), and the margin over text is not small: SoM−DOM is +9.82 pp [+3.57, +16.07] for B0 and +8.04 pp [+3.57, +12.95] for B1 (paired task bootstrap). On WA-reddit the order flips and text leads (SoM−DOM −4.81 pp for B0 and −2.88 pp for B1, both intervals crossing zero), and on VWA-reddit the weakest backbone does worse with the fused mode (−2.96 pp [−5.91, −0.49]), but three of its eight DOM successes were credited although their episodes never visited the target forum, whose subscription an earlier episode had made; without them the gap is −1.48 pp [−3.45, +0.49] (Appendix E). On shopping the fused mode is best or tied with Vision, on the filtered and on the full task sets (Figure 1, Table 4).

Two cautions keep this descriptive. Cells on the same site share their tasks, so the eight are not eight independent replications. And the screenshot-free modes are four, not one: a maximum over four text variants against a single screenshot mode would be a selection effect, so the comparison here takes one representative per deployment class, as practitioners do.

What this section establishes is only that the choice matters and depends on the workload. Whether it can be made per task is the subject of the rest of the paper.

4 The screenshot has a value that can be isolated and, on one site, predicted

A matched contrast. Neither contrast between the deployment choices of §3 isolates the image: SoM−DOM also changes the text payload and the prompt, and Vision−DOM also changes the text and how clicks reach the page (by coordinates, since Vision has no element ids). SoM and P-SoM differ only in the annotated screenshot, so their difference is the value of the image alone. Chi et al. (2024) already compared screenshot-plus-text with text only on a subset of VisualWebArena tasks for one backbone (50.0% against 46.3%, without set-of-marks); we hold the set-of-marks text and prompt fixed, use four backbones and full task sets, rerun the contrast, and ask *where* the image pays off.

An ex-ante rule. A regular expression over the task intent flags tasks that ask about an image, a

photo, a colour or a count of things shown, excluding tasks that already carry a reference image (Appendix D). It reads only the task configuration: no model call and no episode. It was written as a reddit failure-diagnosis pattern a week before it was first used to partition tasks, and its classifieds hits were never inspected, so classifieds is out-of-sample for it.

Result. On the 71 classifieds tasks it flags, the screenshot raises success by +25.35 pp [+14.08, +36.62] for B0, against +5.23 pp on the other 153 tasks (Figure 2). The effect reproduces on a full rerun of both modes (+26.76 pp [+14.08, +39.44], from 23 against 4 successes), and it is larger for the stronger backbones we tested: +2.82 pp for B2, +11.27 pp for B1, +25.35 pp for B0 and +29.58 pp [+16.90, +42.25] for GPT-5.6 (where a parser defect let some actions through beyond the step budget, in both modes alike; Appendix E). The backbones differ in more than capability, so this is an association across the four we ran, not a scaling law. On VWA-reddit the same rule flags 63 tasks and separates nothing: +3.17 pp on flagged tasks against +4.29 pp elsewhere for B0, +1.59 pp against −3.57 pp on its rerun. On shopping the flagged tasks carry at most five successes per mode, too few to read, and on WA-reddit the rule flags only 5 of 104 tasks.

Reading. Where the screenshot pays off on classifieds is predictable before anything runs, and it pays off more for the stronger backbones we tested; on reddit it is not. The screenshot's value is therefore real and partly foreseeable, but it belongs to a kind of task on one site. The rule is also not a router. On classifieds, switching between DOM and Vision by this rule does not beat simply using SoM (for B0, 55 against 61 successes): Vision solves two more flagged tasks than SoM, but DOM solves eight fewer of the rest; what remains open is whether *individual* tasks have stable preferences beyond their difficulty, which is the question of §5.

5 Per-task preferences are reproducible, but one run measures them poorly

§4 found structure at the level of task *types*. Routing per task needs more: that an individual task prefers one representation over another, beyond what an additive model of task difficulty predicts, and that this preference is stable enough to learn from.

A null without task-by-mode structure. In the three cells where every mode was run twice, let $Y_{imr} \in \{0, 1\}$ be the outcome of task i under mode m in run r. Under

$$\operatorname{logit} \Pr(Y_{imr} = 1) = a_i + b_{mr},$$

every task has its own difficulty and every (mode, run) column its own easiness, but there is no persistent task-by-mode interaction. This is the Rasch model, a special case of the multidimensional item-response models that item-response routers fit (Song et al., 2025); here it is the null, and the question is what lies beyond it. The row and column sums are sufficient for this model, so conditional on them every 0/1 matrix with those margins is equally likely; we approximate that conditional null by curveball Markov-chain sampling (Strona et al., 2014), fitting nothing. The statistic is the reproducible part of the interaction, $\hat{\sigma}^2_{\text{int}} = (\text{MS}_{\text{int}} - \text{MS}_{\text{err}})/2$ from the two-way task × mode ANOVA with two runs: the run-to-run covariance of each task's mode profile.

Result. Over the three deployment modes the interaction exceeds the null in all three cells (classifieds B0 $\hat{\sigma}^2_{\text{int}} = 0.033$ against a null 95th percentile of 0.015; VWA-reddit B0 0.018 against 0.009; WA-reddit B1 0.030 against 0.014; $p \leq 0.002$ each, Holm across cells). Over all six modes the same holds (Appendix F). Preferences between representations are therefore a reproducible property of tasks that an additive difficulty model does not capture. The test does not say what the extra structure is: a task-specific preference and a mode whose success responds more steeply to difficulty than another's both reject this null, and only the first is a representation preference in the narrow sense. The preferences also mostly persist where they can be observed directly: over all six modes, taking for each task the cheapest mode that succeeded on one run and scoring that choice on the rerun beats the frontier of the fixed modes and their random mixtures (§6) by 2.4–10.5 points on VWA-reddit B0 and WA-reddit B1, while on VWA-classifieds B0 it lands $+3.57$ or -0.56 points relative to it, depending on which run supplies the choice. That choice uses the task's own label, which no deployment has, so it measures persistence, not an attainable policy.

But a single run measures them poorly. The same decomposition gives, in the sense of a generalisability decision study (Brennan, 2001), the

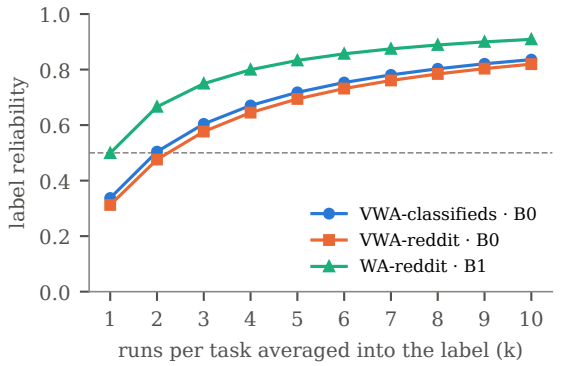


Figure 3: Decision study for the three deployment modes: reliability of a task-by-mode label averaged over k runs, $\sigma^2_{\text{int}}/(\sigma^2_{\text{int}} + \sigma^2_{\text{err}}/k)$. A single run gives 0.31–0.50. Source: `routing_three_arm`.

reliability of a task's mode profile averaged over k runs (Figure 3). From one run it is 0.34, 0.31 and 0.50 in the three cells; the projection crosses 0.5 at two to three runs (bootstrap ranges 2–4, 2–7 and 1–3). Over six modes it is lower still, 0.14–0.46, with up to six runs projected. This is the reliability of the interaction profile, not of a router's chosen arm, so it bounds how noisy single-run training labels are rather than predicting when a router starts to win.

Labels flip most where the choice can change success. A representation choice can change a run's success only on tasks that some modes solve and others do not; on the other tasks it can change only the cost. Over all six modes, the tasks split this way on the first run are 24–39% of each of the three fully rerun cells but 78–84% of the tasks whose outcome changes in some mode on the rerun: 74–86% of the split tasks change, against 6.5–10.3% of the rest. This concentration needs no task-by-mode structure. A binary outcome varies most at intermediate difficulty, and the same no-interaction null over six modes concentrates the changes on these tasks as strongly: the observed ratio of the two change rates, 7.9–11.4, lies between the null's 5th and 95th percentiles in every cell. Nor is task-by-mode structure absent there: these tasks change less often than under the null, below its 5th percentile in every cell.

The routing target thus exists, but a training set built from single runs labels it with a reliability of only 0.31–0.50, and its labels flip most where the choice can change success: in this sense, routing is least learnable where it is most valuable. This compares tasks within a cell, not cells or backbones, and concerns the labels any router learns from; we do not claim that it alone explains the results of §6.

6 Deployable routers do not reliably convert the signal

The comparison a router must win. A router that sends some tasks to an expensive mode and others to a cheap one trades success for cost. Beating every fixed mode on both axes is too strict, but beating the fixed modes one at a time is too lenient: sending a random share of tasks to each of two modes is also a deployable policy. We therefore compare against the *frontier of the fixed modes and all their random mixtures*: the upper concave envelope of the fixed points in the (cost, success) plane, which is the non-decreasing convex hull that model-routing benchmarks and method papers use as a baseline (Hu et al., 2024; Jitkrittum et al., 2025). We report how far above it a router reaches (Figure 4a; definitions in Appendix I).

Routers. Two constructions use 18 features available before a task runs: 14 intent-keyword indicators, the intent length, and three statistics of the first page. *Per-arm heads* fit one logistic success model per mode and send a task to the cheapest mode whose predicted success clears a threshold. *Triage* predicts whether any mode can solve the task and sends predicted-hopeless tasks to the cheapest mode and the rest to the best one. All predictions are out-of-fold (task-held-out five-fold). Both learn from one run's outcomes, which §5 found to measure per-task preferences poorly.

Room above the frontier exists in hindsight. Pooled over cells, the only room that clears a null comes from triage, a decision about difficulty rather than representation, and only under billed cost (see Robustness). Sweeping each router's threshold traces a curve. Over the three deployment modes, the triage curve pooled over the eight cells rises 1.69 points above the frontier at low budget (Figure 4b), which a label-shuffle null that refits the whole curve does not produce ($p = 0.001$; Table 1). The per-arm heads reach 0.86 points ($p = 0.072$). Triage does not look for the representation that suits a task; it decides which tasks are not worth the expensive mode. That one construction passes its null and the other does not is not a test that representation fit carries nothing, since a difference in

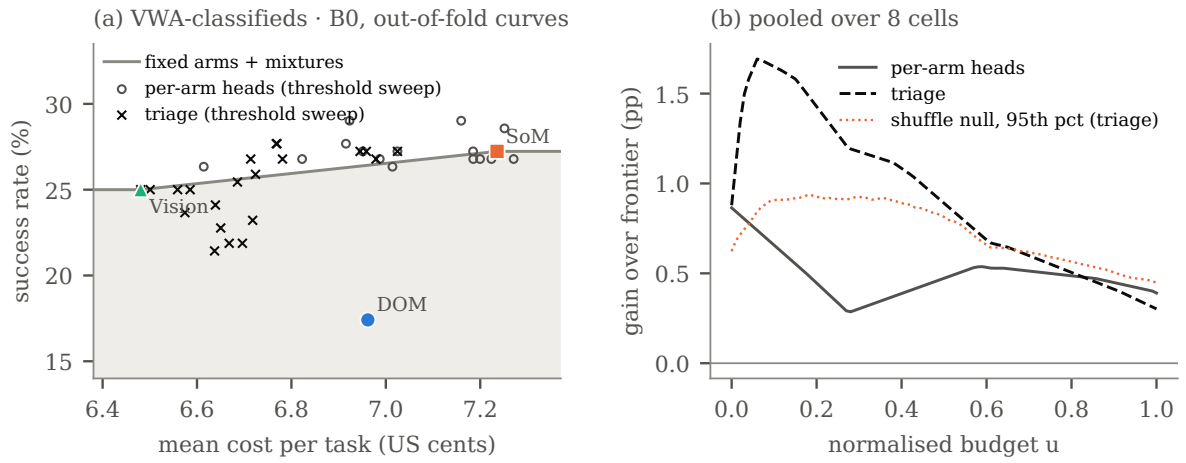


Figure 4: (a) One cell's cost–success plane: the three deployment arms, the frontier of the arms and their random mixtures (shaded: attainable without routing), and the out-of-fold points of the two routers as their threshold is swept. (b) Gain of the frontier when each router's curve is added, averaged over the eight cells on a budget normalised so that the cheapest arm is $u = 0$ and the dearest $u = 1$, against the pointwise 95th percentile of the triage curve's label-shuffle null, which refits the whole curve (the p-values in Table 1 test the maximum over u). Both panels choose the best point after seeing test outcomes; Table 1 gives the deployable version. Source: `routing_three_arm`.

significance is not a significant difference.

A deployable router does not capture it. A curve's best point is chosen after seeing test outcomes. When the operating point is instead chosen on training folds only, the pooled gain falls to +0.23 points for per-arm heads and +0.59 for triage, against 0.86 and 1.69 in hindsight (Table 1). The per-arm interval has its upper end below its point estimate, a sign that the task bootstrap of a policy whose threshold is re-selected in every draw is biased; we therefore read all deployable intervals as magnitudes and make no significance claim from them, in either direction. Over all six modes both deployable routers sit slightly below the frontier (-0.38 and -0.39). The pattern holds for the strongest backbone: for GPT-5.6 on classifieds, where the screenshot's value was largest (§4), the deployable routers lose 1.63–2.45 points; on both shopping cells, filtered or on all tasks, they stay within 1.3 points of the frontier.

Robustness. Holding tasks of the same template out together lowers the six-mode per-arm router's mean deployable gain from -0.38 to -1.60 points, and fitting on one run and scoring on its rerun, with the same template folds, gives point estimates of both signs (Appendix G). Repricing cost changes the hindsight result but not the deployable one (Table 3). Under wall-clock cost the pooled triage curve no longer clears its null (+0.68 points, $p = 0.42$), nor does it under GPU time on the five locally served cells (+1.24, $p = 0.39$); the per-arm heads stay just short of significance under every basis ($p = 0.072$–0.077). Over all cells of each basis the deployable gains stay between -0.5 and $+1.5$ points. Dropping the two cells of the weakest backbone (B2) leaves the billed picture unchanged (triage curve +1.85, $p = 0.001$; deployable +0.50 and +0.39).

7 A protocol for multi-representation claims

The three quantities of §1 came apart because each was measured against its own control. We suggest four steps for any claim that choosing among several agent configurations per task would pay.

1. **Report reruns per arm and per cell**, not one flip rate for a whole study: in our data a rerun changes between 0 and 14% of outcomes depending on arm and cell (Appendix F).
2. **Isolate the component of interest with a matched ablation** (here SoM−P-SoM, which removes only the screenshot) before attributing a difference to it.
3. **Test for per-task structure against a null that conditions on difficulty**, and use a decision study to say how many runs per task

cells	policy	curve max (p)	deployable [5%, 95%]
3 arms, 8 cells pooled	per-arm heads	+0.86 (0.072)	+0.23 [−2.20, +0.22]
	triage	+1.69 (0.001)	+0.59 [−1.53, +1.23]
6 arms, 8 cells pooled	per-arm heads	+0.67 (0.252)	−0.38 [−3.20, −0.14]
	triage	+1.24 (0.043)	−0.39 [−2.42, +0.81]
GPT-5.6, classifieds (5 arms)	per-arm heads	+0.45 (0.768)	−2.45 [−8.61, +4.24]
	triage	+3.86 (0.127)	−1.63 [−6.70, +5.74]
Shopping B0, clean (3 arms)	per-arm heads	+1.14 (0.062)	−0.18 [−2.45, +2.02]
	triage	+0.20 (0.496)	−0.87 [−2.63, +1.37]
Shopping B0, all tasks (3 arms)	per-arm heads	+1.09 (0.049)	+1.30 [−2.31, +1.85]
	triage	+0.18 (0.541)	−0.05 [−2.14, +1.45]
Shopping B1, clean (6 arms)	per-arm heads	+1.11 (0.143)	+0.56 [−4.69, +0.88]
	triage	+1.63 (0.158)	+1.19 [−4.10, +1.57]
Shopping B1, all tasks (6 arms)	per-arm heads	+0.48 (0.169)	−0.43 [−2.78, +0.73]
	triage	+0.69 (0.289)	+0.40 [−2.02, +2.02]

Table 1: Routing gain over the fixed arms and their random mixtures, in success-rate points. *Curve max*: the best point of an out-of-fold curve, chosen after seeing test outcomes, with its label-shuffle p (pooled rows: max over a normalised budget). *Deployable*: the operating point chosen on training folds only, task bootstrap. Sources: `routing_three_arm`, `representation_routing_frontier`, `routing_extension_cells`.

make the label reliable. A hindsight ceiling from one run is not one a router can reach.

4. **Score a deployable policy, with its operating point chosen on training data, against the fixed arms and their random mixtures**, never only against the best single arm or a hindsight curve.

Applied here, the protocol says that the screenshot raises success most on a predictable class of classifieds tasks, and that per-task representation preferences are reproducible but single-run labels measure them with a reliability of only 0.31–0.50; over six modes those labels also flip most where the choice can change success, as the additive difficulty model predicts, which is the sense in which routing is least learnable where it is most valuable. With single-run labels and pre-flight features, the routers we tested do not reliably beat choosing one representation, or a fixed mixture, for the whole workload.

8 Related work

How web agents observe the page. Agents have read HTML or accessibility trees (Deng et al., 2023; Yang et al., 2025), screenshots grounded to elements (Zheng et al., 2024; He et al., 2024; Gou et al., 2025), or set-of-marks overlays that fuse the two (Yang et al., 2023; Koh et al., 2024). A parallel line shrinks or reshapes the text observation (Kerboua et al., 2025; Schiepanski and Piël, 2025; Enomoto et al., 2026), and some systems combine several grounding routes (Li et al., 2026a). OSWorld compares screenshot, accessibility-tree, combined and set-of-marks inputs across models and finds the best input model-dependent (Xie et al., 2024), and Chi et al. (2024) compare screenshot-plus-text with text only on a VisualWebArena subset. Closer to adaptivity, Li and Shi (2026) gate visual observation components during execution of computer-use tasks, and Liu et al. (2026) route each action of a computer-use agent among vision-language models; neither chooses between text and screenshot per task before it runs. These works compare observation designs at the level of a benchmark score or adapt them online. Our contribution is not that the best observation varies, which OSWorld already shows, but a matched removal of the screenshot, repeated task-level outcomes, a test of task structure beyond difficulty, and a deployable per-task choice judged against mixtures of the fixed choices.

Routing and cascades. Choosing among models per query to trade quality for cost is well studied for language models (Chen et al., 2024; Ong et al., 2025; Ding et al., 2024; Gupta et al., 2024; Peale et al., 2026); see Moslem and Kelleher (2026) for a survey. For web agents, Li et al. (2025) learn query-specific routing for cost-sensitive agents, and Hajimiri et al. (2026) judge agent components at a matched budget. Early exit (Lu et al., 2025) stops agents that are unlikely to succeed. That is the difficulty lever, which in our data is the only part of routing whose pooled hindsight room above the fixed frontier clears its null. Routing benchmarks

and method papers compare routers against random mixtures of fixed models (Hu et al., 2024; Jitkrittum et al., 2025), and under unified conditions several learned routers fail to beat a simple baseline (Li et al., 2026b); Lee (2026) finds that assigning one model per task type in advance recovers most of the oracle routing gain on a suite of seven task types in three languages, and that reruns change the score of 5% of model–question pairs. Item-response routers model query difficulty and model ability (Song et al., 2025). We route over observation representations rather than models, take the mixture baseline from this literature, and add what it rarely has: reruns of identical agent conditions and a test of per-task structure beyond difficulty.

Reliability of agent evaluation. Repeated trials of the same agent disagree, which τ-bench measures with pass^k (Yao et al., 2025); inference itself is not deterministic even at fixed settings (He and Thinking Machines Lab, 2025; Yuan et al., 2025), prompts change outcomes in ways unrelated to content (Sclar et al., 2024), and agent benchmarks carry evaluator and task defects (El Hattami et al., 2025; Bhat et al., 2026). Our reruns quantify the first for web agents at the task level and show what it does to per-task routing labels; the harness defects we found are listed in Appendix E.

Limitations

Coverage of reruns. Only three cells have every mode run twice, two with the same backbone; the preference test and decision study (§5) rest on them. Of the remaining eight of the eleven cells, five have some modes rerun and three have none. On classifieds B2 solves about 2% of tasks, so its two reruns cannot bound an effect.

Breadth. VWA-reddit and WA-reddit are task sets on the same application, so the study covers three applications, not four benchmarks. GPT-5.6 was run on classifieds only, and its Vision run is excluded for a coordinate-contract defect; a parser defect also let it take actions beyond the 30-step budget in every mode (Appendix E). The ex-ante rule of §4 works on classifieds only.

Shopping filtering. The clean shopping cells drop every task on which the search-box defect fired in any mode, as well as the 42 grid-order-sensitive and 4 wishlist tasks (Appendix E). Whether it fired depends on what the agent typed, so the filter selects on behaviour under the conditions being compared: the clean cells estimate the comparison on an observed defect-free subset, not on the benchmark, and B0 and B1 keep different subsets (368 and 192 tasks). Results on all tasks are reported alongside (Tables 1 and 4); no conclusion rests on the filtered cells alone.

What the routing result does not show. The routers use 18 pre-flight features and logistic models; richer features, language-model routers or multiple runs per training task may do better. The result is that, with single-run labels and these features, the routers do not reliably beat the fixed choices and their mixtures. It does not show that per-task routing is impossible. Step-level switching is out of scope.

Inherited state. Reddit and shopping are reset before each run but not between tasks, so an episode can be credited for state an earlier episode created. A mechanical audit removes reddit successes credited without visiting the target forum and changes one comparison in the paper (§3); it cannot see an episode that visits a forum an earlier episode already subscribed to and leaves without acting, so reddit outcomes may still carry inherited credit, and reruns may reproduce it.

Exploratory analyses. The preference test, the reading of label changes in §5, the frontier comparisons and the deployable-router estimates were designed after the runs were collected. Only the preference test is corrected across cells (Holm); the frontier p-values account for choosing each curve's best point but are not corrected across curves (Holm across the 14 curve maxima of Table 1 keeps only the three-mode pooled triage curve); the bootstrap intervals for policies with a selected threshold are read as magnitudes, not tests.

Cost. API charges and local-serving estimates are different quantities and are never pooled, and local cost depends on the pricing basis: the pooled three-mode triage curve clears its null under billed cost but not under wall-clock time or, on the locally served cells, GPU time (§6).

Ethics statement

All episodes ran against self-hosted copies of the VisualWebArena and WebArena sites; no real account or person was contacted (but agents could follow external links off the self-hosted sites: in

the audited runs, 3–6% of reddit episodes and at most 0.3% of classifieds episodes visited live public pages), and no human subjects were involved. Sites were reset before each run. The API-served backbones were used under their providers' terms. Agents that act on websites can cause harm when deployed against live services; this study measures observation choices on self-hosted sites and releases no agent that acts outside them.

Availability. The anonymous supplementary material contains the agent and run configurations, per-episode outcomes and costs for every run used, the analysis scripts, and every analysis product cited in the text, with a guide that maps each number in the paper to the product field it comes from; figures and tables are regenerated from the products by one script. Full trajectories (screenshots and page text) are too large for the supplement and will be released with the camera-ready version.

References

Vishvesh Bhat, Jay Vaghasiya, Muhammad Ahmed Mohsin, and Asad Aali. 2026. Benchmarking the Benchmarks: A Validity Audit of Tool-Calling Evaluation. *Preprint*, arXiv:2607.02577.

Robert L. Brennan. 2001. *Generalizability Theory*. Springer, New York.

Lingjiao Chen, Matei Zaharia, and James Zou. 2024. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research*.

Wayne Chi, Ameet Talwalkar, and Chris Donahue. 2024. The Impact of Element Ordering on LM Agent Performance. *Preprint*, arXiv:2409.12089.

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*.

Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Rühle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. 2024. Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing. In *International Conference on Learning Representations*.

Amine El Hattami, Megh Thakkar, Nicolas Chapados, and Christopher Pal. 2025. WebArena Verified: Reliable Evaluation for Web Agents. In *Workshop on Scaling Environments for Agents*. NeurIPS 2025 workshop; also see https://github.com/ServiceNow/webarena-verified.

Masafumi Enomoto, Ryoma Obara, Haochen Zhang, and Masafumi Oyamada. 2026. Revisiting Observation Reduction for Web Agents: Comprehensive Evaluation with a Lightweight Framework. *Preprint*, arXiv:2605.29397.

Boyu Gou, Demi Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. 2025. Navigating the Digital World as Humans Do: Universal Visual Grounding for GUI Agents. In *International Conference on Learning Representations*.

Neha Gupta, Harikrishna Narasimhan, Wittawat Jitkrittum, Ankit Singh Rawat, Aditya Krishna Menon, and Sanjiv Kumar. 2024. Language Model Cascades: Token-Level Uncertainty and Beyond. In *International Conference on Learning Representations*.

Sina Hajimiri, Masih Aminbeidokhti, Jose Dolz, Ismail Ben Ayed, Issam H. Laradji, Spandana Gella, and Nicolas Gontier. 2026. Are Online Skill and Memory Modules Always Worth Their Tokens? A Budget-Constrained Study of Web Agents. *Preprint*, arXiv:2606.15017.

Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024. WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.

Horace He and Thinking Machines Lab. 2025. Defeating Nondeterminism in LLM Inference. Thinking Machines Lab blog.

Qitian Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Upadhyay. 2024. RouterBench: A Benchmark for Multi-LLM Routing System. In *Agentic Markets Workshop at ICML 2024*.

Wittawat Jitkrittum, Harikrishna Narasimhan, Ankit Singh Rawat, Jeevesh Juneja, Congchao Wang, Zifeng Wang, Alec Go, Chen-Yu Lee, Pradeep Shenoy, Rina Panigrahy, Aditya Krishna Menon, and Sanjiv Kumar. 2025. Universal Model Routing for Efficient LLM Inference. *Preprint*, arXiv:2502.08773.

Imene Kerboua, Sahar Omidi Shayegan, Megh Thakkar, Xing Han Lù, Léo Boisvert, Massimo Caccia, Jérémy Espinas, Alexandre Aussem, Véronique Eglin, and Alexandre Lacoste. 2025. FocusAgent: Simple Yet Effective Ways of Trimming the Large Context of Web Agents. *Preprint*, arXiv:2510.03204.

Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. 2024. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.

Janghoon Lee. 2026. Most of the LLM Routing Gap Is Task Type. *Preprint*, arXiv:2608.23023.

Aiden Yiliu Li, Xinyue Hao, Shilong Liu, and Mengdi Wang. 2026a. Avenir-Web: Human-Experience-Imitating Multimodal Web Agents with Mixture of Grounding Experts. *Preprint*, arXiv:2602.02468.

Bojie Li and Noah Shi. 2026. Agent-Computer Observation Interfaces Enable Dynamic Computer Use. *Preprint*, arXiv:2606.29472.

Hao Li, Yiqun Zhang, Zhaoyan Guo, Chenxu Wang, Shengji Tang, Qiaosheng Zhang, Yang Chen, Biqing Qi, Peng Ye, Lei Bai, Zhen Wang, and Shuyue Hu. 2026b. LLMRouterBench: A Massive Benchmark and Unified Framework for LLM Routing. In *Findings of the Association for Computational Linguistics: ACL 2026*.

Tao Li, Jinlong Hu, Yang Wang, Junfeng Liu, and Xuejun Liu. 2025. WebRouter: Query-specific Router via Variational Information Bottleneck for Cost-sensitive Web Agent. *Preprint*, arXiv:2510.11221.

Xunzhuo Liu, Bowei He, Xue Liu, Andy Luo, Haichen Zhang, and Huamin Chen. 2026. Adaptive Vision-Language Model Routing for Computer Use Agents. *Preprint*, arXiv:2603.12823.

Qingyu Lu, Liang Ding, Siyi Cao, Xuebo Liu, Kanjian Zhang, Jinxia Zhang, and Dacheng Tao. 2025. Runaway is Ashamed, But Helpful: On the Early-Exit Behavior of Large Language Model-based Agents in Embodied Environments. In *Findings of the Association for Computational Linguistics: EMNLP 2025*.

Yasmin Moslem and John D. Kelleher. 2026. Dynamic Model Routing and Cascading for Efficient LLM Inference: A Survey. *Transactions on Machine Learning Research*. Survey Certification.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2025. RouteLLM: Learning to Route LLMs from Preference Data. In *International Conference on Learning Representations*.

Charlotte Peale, Siddartha Devic, Parikshit Gopalan, Udi Wieder, and Aravind Gollakota. 2026. Flexible Routing via Uncertainty Decomposition. *Preprint*, arXiv:2605.07805. ICML 2026 Workshop on Statistical Frameworks for Uncertainty in Agentic Systems.

Thassilo M. Schiepanski and Nicholas Piël. 2025. Beyond Pixels: Exploring DOM Downsampling for LLM-Based Web Agents. *Preprint*, arXiv:2508.04412.

Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. 2024. Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I learned to start worrying about prompt formatting. In *International Conference on Learning Representations*.

Wei Song, Zhenya Huang, Cheng Cheng, Weibo Gao, Bihan Xu, GuanHao Zhao, Fei Wang, and Runze Wu. 2025. IRT-Router: Effective and Interpretable Multi-LLM Routing via Item Response Theory. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15629–15644.

Giovanni Strona, Domenico Nappo, Francesco Boccacci, Simone Fattorini, and Jesus San-Miguel-Ayanz. 2014. A Fast and Unbiased Procedure to Randomize Ecological Binary Matrices with Fixed Row and Column Totals. *Nature Communications*, 5:4114.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024) Datasets and Benchmarks Track*.

Jianwei Yang, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. 2023. Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V. *Preprint*, arXiv:2310.11441.

Ke Yang, Yao Liu, Sapana Chaudhary, Rasool Fakoor, Pratik A. Chaudhari, George Karypis, and Huzefa Rangwala. 2025. AgentOccam: A Simple Yet Strong Baseline for LLM-Based Web Agents. In *International Conference on Learning Representations*.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *International Conference on Learning Representations*.

Jiayi Yuan, Hao Li, Xinheng Ding, Wenya Xie, Yu-Jhe Li, Wentian Zhao, Kun Wan, Jing Shi, Xia Hu, and Zirui Liu. 2025. Understanding and Mitigating Numerical Sources of Nondeterminism in LLM Inference. In *Advances in Neural Information Processing Systems*.

Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. 2024. GPT-4V(ision) is a Generalist Web Agent, if Grounded. In *Proceedings of the 41st International Conference on Machine Learning*.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations*.

A Task universes and exclusions

Scored sets are the collected tasks minus protocol exclusions written as rules over task configuration

or, for one reddit task, over agent trajectories: classifieds 224 of 224; VWA-reddit 203 of 205 (one task whose evaluator never checks the requested action, and one cross-site task answerable from memory); shopping 432 of 435. For every site the collected tasks are the benchmark's tasks minus its unachievable tasks (reference answer "N/A"), which we drop because our agent prompt does not offer N/A as an answer (WA-reddit: 104 of 106). Every analysis records the task set it was built on and fails if denominators differ.

B Agent and run contract

Every condition uses the same agent loop and settings; only the observation mode and backbone vary, apart from one output-format line of the system prompt (B0 is told to answer through a tool call, GPT-5.6 to emit exactly one JSON object). Decoding is greedy (temperature 0, at most 4096 new tokens per step). The browser viewport is 1280×720, and the page is observed 0.5 s after each action. The prompt holds the task instruction, the mode's system prompt, the last eight steps (action, outcome and resulting URL) and the current observation; the text observation (accessibility tree or marked-element list) covers only the current viewport and is never truncated. The nominal step budget is 30; three consecutive unparsable outputs end an episode. B1 (Qwen3-VL-4B-Instruct) and B2 (Gemma-3-4B-it) run locally from pinned model revisions; B0 (Qwen3-VL-235B-A22B) and B5 (the mid-price GPT-5.6 tier) are reached through a hosted API under a fixed model identifier. Each run uses seed 42, starts from freshly reset sites, and records the code version, configuration and environment with every episode. The exact prompts and configuration files are in the supplementary material.

C The six observation modes

Four modes carry no page screenshot and cross two factors: the text payload (the accessibility tree, or the flattened list of set-of-marks elements) and the prompt family (DOM or SoM). DOM is tree + DOM prompt; P-prompt is tree + SoM prompt; P-text is marks + DOM prompt; P-SoM is marks + SoM prompt. SoM adds the annotated screenshot to P-SoM; Vision is the raw screenshot with an empty text payload. A task's own reference image, when it has one, is given in every mode.

D The intent rule

A task is flagged if its intent matches

```
\b(image|picture|photo|screenshot)\b|
\bcolou?r of\b|
\bhow many\b[^.]{0,40}\bin (?:the|this)\b
```

(case-insensitive) and the task carries no reference image. It flags 71 of 224 classifieds tasks, 63 of 203 VWA-reddit tasks, 56 of 432 shopping tasks and 5 of 104 WA-reddit tasks. It was written while auditing reddit failures; classifieds is out-of-sample.

E Harness defects and how cells were filtered

- **GPT-5.6 Vision**: the run used a broken coordinate contract and is excluded.
- **GPT-5.6 step budget**: when the model emitted several valid actions, the parser executed one but booked the step as a free wait. Summed over a whole condition, GPT-5.6 had 49–89 such steps (1.2–2.4% of its steps; most of them executed an action, and some episodes ran 31–32 steps in all), B2 had 2–24, and B0 and B1 none (counts from the classifieds runs). It affects SoM and P-SoM alike.
- **Reddit state**: subscriptions persist across the episodes of a run. Of the reddit successes scored by the subscribed-forum list, six were credited although their episodes never visited the target forum, three of them for B2 with DOM; §3 reports the B2 comparison with and without them. The audit cannot detect an episode that reaches an already-subscribed forum and leaves without acting (one such case is confirmed on WA-reddit).
- **Shopping search box**: typing into a search box that already holds a query submitted old + new text ("chair" + "chair with wheels" → "chairchair with wheels") in 43% of such submissions, far more often in text modes than in Vision. A detector reproducing the defect's per-mode counts marks affected episodes; the *clean* shopping cells drop every task affected in any mode (18 tasks for B0, 201 for B1).
- **Shopping state**: the category grid order differs between runs (42 order-sensitive tasks dropped), and the wishlist is not reset between tasks (4 tasks dropped); with all three filters, B0 keeps 368 of 432 tasks and B1 192.

F Six-mode results

Table 2 gives the success rate of every mode in every cell. Over all six modes, the no-interaction null of §5 is rejected in all three fully rerun cells ($p = 0.0005$, 0.016, 0.0005; Holm), with single-run reliability 0.29, 0.14 and 0.46 and three, six and two runs per task needed to reach 0.5. Across the 27 registered rerun pairs, a rerun changes between 0 and 14% of task outcomes, depending on arm and cell.

G Template and rerun robustness

Averaged over cells, the deployable excess over the frontier with folds grouped by task template is -1.60 (per-arm heads) and -0.84 (triage) points over six modes, and -0.64 and $+0.29$ over three. Fitting on one run and scoring on its rerun, with the same template folds and averaged over the three fully rerun cells, gives -0.90 and -1.60 over six modes and -0.60 and -0.56 over three. Over six modes, every per-cell template-bootstrap interval (90%) includes zero; the three-mode readings are point estimates.

H Per-cell results

Table 4 lists the success rate and cost behind Figures 1 and 4. In shopping the fused mode is cheaper than text: its marked-element list is shorter than the accessibility tree of a product-dense page.

I Estimators

Frontier. For a cell with arms m of mean cost c_m and success rate s_m, $F(c)$ is the upper concave envelope of $\{(c_m, s_m)\}$, cut at the highest-success arm (constant beyond it) and equal to the cheapest arm's success below the cheapest cost. Between the cheapest and the highest-success arm, every point of F is reached by sending a random share of tasks to each of two arms; no mixture costs less than the cheapest arm, so F below that cost is a convention that favours the fixed arms. A policy with mean cost c and success s has excess $s - F(c)$.

Router curves. Features are standardised within each training fold and fed to an L2-regularised logistic regression ($C = 1$); predictions are out-of-fold over five folds of distinct tasks. *Per-arm heads*: for each threshold $\tau \in \{0, 0.05, \ldots, 1\}$ a task goes to the cheapest arm (by training-fold mean cost) whose predicted success is at least τ, and to the training fold's best arm if none is. *Triage*: one model predicts whether any arm solves the task; for each quantile $q \in \{0, 0.05, \ldots, 1\}$ of its score, tasks below the quantile go to the cheapest arm and the rest to the best arm.

Pooled gain. Within a cell, cost is normalised as $u = (c - c_{\min})/(c_{\max} - c_{\min})$ over the fixed arms, which makes cells with different cost units comparable without pooling their costs. The gain at u is the envelope of the fixed arms together with the curve's points minus the envelope of the fixed arms alone, on 101 grid points; the pooled curve is the equal-weight mean over cells and its statistic is the maximum over u. The null permutes each task's outcomes and costs against its features and refits the whole curve ($B = 1000$; $p = (k+1)/(B+1)$; for the pooled statistic each null draw is averaged over cells before the maximum).

Deployable policy. In each outer fold the threshold (or quantile) is the one with the largest excess over the *training* tasks' own frontier, using in-sample predictions on those tasks; it is then applied to the held-out fold. The five folds' decisions form one policy over all tasks, scored against the full-sample frontier. Intervals resample tasks and rerun the whole pipeline ($B = 1000$, 5th and 95th percentiles); pooled intervals average the cells draw by draw.

No-interaction null and decision study. For the task $\times$ mode $\times$ run array with $r = 2$ runs, the two-way random-effects ANOVA gives $\hat{\sigma}^2_{\text{err}} = \text{MS}_{\text{err}}$ and $\hat{\sigma}^2_{\text{int}} = (\text{MS}_{\text{int}} - \text{MS}_{\text{err}})/r$. The null matrices are drawn with curveball trades over the task-by-(mode, run) 0/1 matrix, which preserve every row and column total (four chains, each with 50 trades per row of burn-in and 5 per row between kept draws, 2000 draws in all; for the six-mode analysis a second sampler with four-fold thinning checks mixing). The reliability of a label averaged over k runs is $\hat{\sigma}^2_{\text{int}}/(\hat{\sigma}^2_{\text{int}} + \hat{\sigma}^2_{\text{err}}/k)$, and the number of runs needed to reach 0.5 is $\lceil \hat{\sigma}^2_{\text{err}}/\hat{\sigma}^2_{\text{int}} \rceil$.

		no screenshot				screenshot	
cell	n	DOM	P-prompt	P-text	P-SoM	SoM	Vision
cls B0	224	17.4†	19.6†	15.6†	15.6†	27.2†	25.0†
cls B1	224	6.2†	6.7	7.6	6.7	14.3†	12.5†
cls B2	224	1.3	1.8	0.4	0.9	2.2†	2.2†
VWA-red B0	203	14.3†	12.3†	13.3†	10.8†	14.8†	7.4†
VWA-red B1	203	5.9†	5.4	5.9	5.9	7.4†	2.5
VWA-red B2	203	3.9	0.0	2.0	0.5	1.0	2.0
WA-red B0	104	26.9	26.0	35.6	25.0	22.1	19.2
WA-red B1	104	16.3†	16.3†	16.3†	11.5†	13.5†	9.6†
cls B5	224	23.7†	21.9	24.1	22.8	37.1	–
shop B0 clean	368	12.8	–	–	–	16.8	16.6
shop B0 all	432	11.3	–	–	–	14.8	14.4
shop B1 clean	192	5.7	7.3	6.8	5.2	9.4	6.8
shop B1 all	432	4.9	5.3	6.7	4.6	7.6†	5.6

Table 2: Success rate (%) of all six observation modes per cell (Appendix C); † marks an arm with a registered same-condition rerun (Appendix F). Shopping B0 was not run with P-prompt, P-text or P-SoM; GPT-5.6 Vision is excluded; shopping rows as in Table 4. Sources: `representation_routing_frontier`, `routing_extension_cells`, `noise_floor_inventory`.

		curve max (p)		deployable	
cost	cells	per-arm	triage	per-arm	triage
billed	all 8	+0.86 (.072)	+1.69 (.001)	+0.23	+0.59
	no B2 6	+1.15 (.071)	+1.85 (.001)	+0.50	+0.39
wall-clock	all 8	+1.28 (.077)	+0.68 (.418)	+0.10	−0.48
	no B2 6	+1.70 (.076)	+0.60 (.619)	+0.18	−0.95
GPU time	all 5	+1.61 (.076)	+1.24 (.387)	+1.47	+0.05
	no B2 3	+2.68 (.076)	+2.06 (.376)	+2.58	−0.56

Table 3: Three-mode routing pooled over cells under three cost bases, with and without the weakest backbone (B2). GPU time exists only for the locally served B1 and B2. Curve max: best point after seeing test outcomes, with its label-shuffle p; deployable: operating point chosen on training folds. Sources: `routing_three_arm` and its `_wallclock`, `_gpu_time` siblings.

		DOM		SoM		Vision	
cell	n	SR	cost	SR	cost	SR	cost
cls B0	224	17.4	6.96	27.2	7.24	25.0	6.48
cls B1	224	6.2	5.95	14.3	6.03	12.5	4.32
cls B2	224	1.3	7.68	2.2	9.08	2.2	7.07
VWA-red B0	203	14.3	10.15	14.8	11.05	7.4	9.81
VWA-red B1	203	5.9	7.33	7.4	8.00	2.5	5.24
VWA-red B2	203	3.9	9.48	1.0	11.16	2.0	6.83
WA-red B0	104	26.9	7.53	22.1	9.11	19.2	8.64
WA-red B1	104	16.3	6.58	13.5	7.94	9.6	4.47
cls B5	224	23.7	13.74	37.1	16.71	–	–
shop B0 clean	368	12.8	12.25	16.8	9.87	16.6	7.75
shop B0 all	432	11.3	11.99	14.8	9.77	14.4	7.48
shop B1 clean	192	5.7	10.60	9.4	7.75	6.8	4.69
shop B1 all	432	4.9	11.11	7.6	8.27	5.6	4.94

Table 4: Success rate (%) and mean billed cost per task (US cents) of the three deployment arms. B0 and GPT-5.6 (B5) costs are API charges computed from token counts and per-token prices; B1 and B2 costs are token-priced estimates for local serving, comparable within a cell only. Shopping: clean = tasks on which the search-box defect fired in no mode, also excluding the 42 grid-order-sensitive and 4 wishlist tasks (Appendix E), all = every scored task; GPT-5.6 Vision is excluded. Sources: `routing_three_arm`, `routing_extension_cells`.